

Séance 3 : TP N°1 — Contrôle de saisie & Boucle Tant Que sur Machine

Évaluation pratique sur machine N°1 (TP noté individuel) sur les boucles itératives non bornées et le contrôle de saisie

1. Consignes Générales

 **Organisation du TP :** Durée 90 minutes. Travail individuel sur machine.

Créez sur votre bureau le script Python `TP1_Nom_Prenom.py`. Rédigez les algorithmes et les TDO demandés sur votre feuille de copie.

Respect des Conventions	Trace Écrite (Copie)	Validation Machine
Noms de variables explicites, types stricts, pas de conversion abusive en chaîne, interdiction formelle de l'instruction <code>break</code> .	Rédigez sur votre feuille les algorithmes normalisés en pseudo-code et les Tableaux de Déclaration des Objets (TDO).	Testez chaque activité avec les jeux d'essais obligatoires avant de faire valider le fonctionnement par l'enseignant.

2. Palier Socle de Réussite (8 points)

Sécuriser la saisie du code d'accès opérateur d'une machine-outil et analyser sa structure par divisions successives.

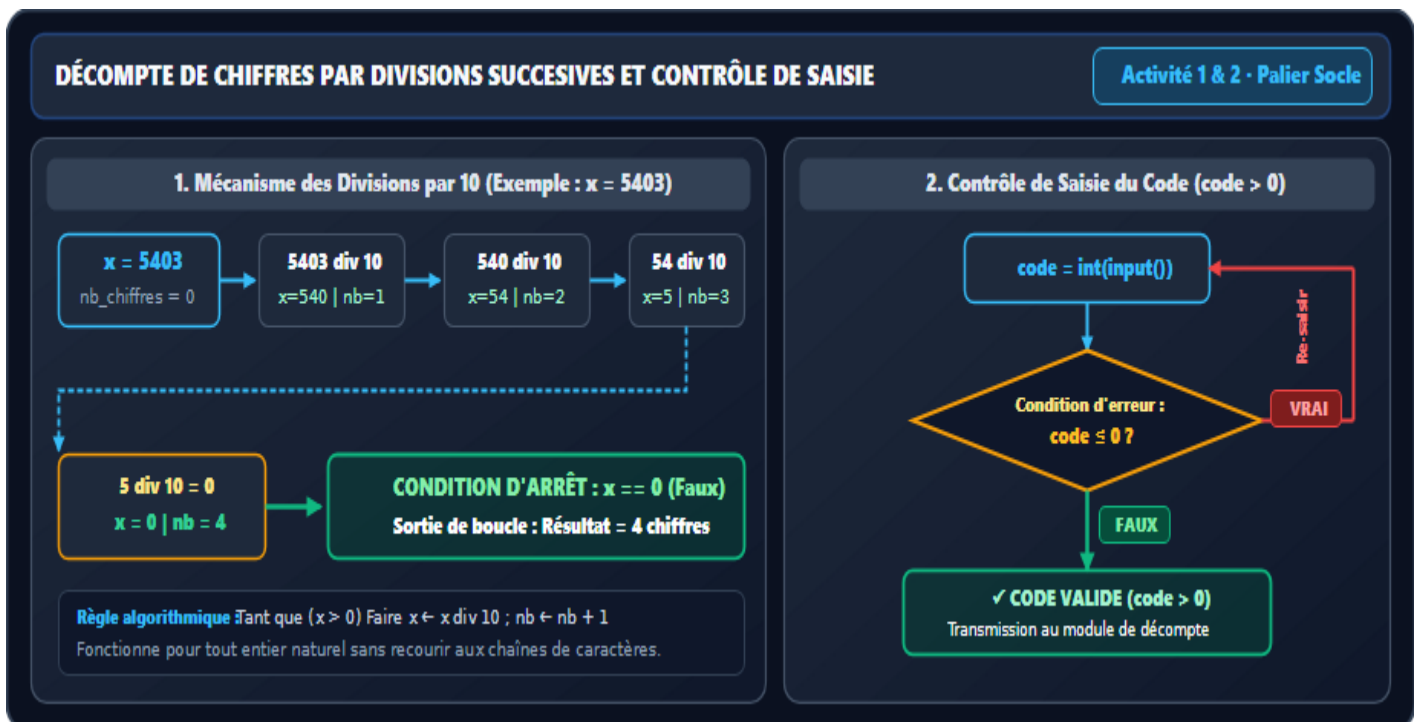


Figure 1 : Décompte des chiffres par divisions successives par 10 et patron de saisie contrôlée.

Activité 1 — Contrôle d'accès sécurisé à une machine-outil (MOCN) 4 points

Mise en situation technique & Cahier des charges

Sur une machine-outil à commande numérique (MOCN), l'accès au pupitre de commande est verrouillé. Pour s'authentifier, l'opérateur doit introduire un code numérique strictement positif ($code \in \mathbb{N}^*$).

Le module de sécurité exécute un traitement séquentiel en trois étapes indissociables :

1. **Étape 1 — Saisie filtrée obligatoire** : Le système invite l'utilisateur à saisir `code`. Tant que la valeur entrée est invalide ($code \leq 0$), le système affiche « *Code invalide ! Entrez un entier strictement positif.* » et réitère la demande.
2. **Étape 2 — Analyse du format & Décompte des chiffres** : Une fois le code validé, le système détermine son nombre exact de chiffres par **divisions entières successives par 10** ($copie_code \div 10$), sans jamais recourir à la conversion en chaîne de caractères (interdiction formelle de `ConvCh()` ou `Long()`).
3. **Étape 3 — Décision d'habilitation** :
 - Si le code comporte **4 chiffres** $\Rightarrow$ Afficher « *Accès Technicien Régleur autorisé* ».
 - Si le code comporte **3 chiffres** $\Rightarrow$ Afficher « *Accès Opérateur Usinage autorisé* ».
 - Pour tout autre format $\Rightarrow$ Afficher « *Format de code non accrédité : Accès refusé !* ».

Principe algorithmique & Validation (Exemple : code = 5403)

```
// Initialisation : copie_code  $\leftarrow$  code (5403), nb  $\leftarrow$  0
// Étape 1 : 5403 div 10 = 540 -> nb = 1
// Étape 2 : 540 div 10 = 54 -> nb = 2
// Étape 3 : 54 div 10 = 5 -> nb = 3
// Étape 4 : 5 div 10 = 0 -> nb = 4
// Arrêt : copie_code = 0 -> Résultat = 4 chiffres
// Habilitation : Profil « Technicien Régleur »
```

Jeux d'essais obligatoires :

- $code = 5403 \Rightarrow$ 4 chiffres $\Rightarrow$ Technicien Régleur
- $code = 176 \Rightarrow$ 3 chiffres $\Rightarrow$ Opérateur Usinage
- $code = 9 \Rightarrow$ 1 chiffre $\Rightarrow$ Format non accrédité
- $code = -45 \Rightarrow$ Erreur et répétition de saisie

Sur votre copie : Rédigez l'algorithme complet sur votre feuille et enregistrez le script sous `TP1_Nom_Prenom.py`.

Activité 2 — Tableau de Déclaration des Objets & Justification des choix 4 points

Dresser le Tableau de Déclaration des Objets (TDO) de l'Activité 1 sur votre feuille d'examen :

Objet	Type / Nature	Rôle & Justification du type
code		Code d'accès saisi par l'opérateur (Entier naturel ≥ 1)
copie_code		Variable de travail entière préservant la valeur initiale de code lors des divisions successives
nb_chiffres		Compteur d'itérations / nombre total de chiffres du code (Entier ≥ 0)

1. Pourquoi est-il indispensable de copier la valeur de code dans copie_code avant la boucle ? Que se passerait-il si on effectuait les divisions directement sur la variable code ?
2. Pourquoi l'opérateur div (division entière) est-il impératif à la place de l'opérateur / (division réelle) ? Quel serait l'impact sur la condition d'arrêt ?
3. Pourquoi l'usage de la fonction Long(ConCh(code)) est-il proscrit dans cette épreuve d'algorithmique fondamentale ?

Sur votre copie : Complétez le TDO et justifiez vos choix de typage sur votre feuille d'évaluation.

3. Palier Maîtrise du Niveau Attendu (8 points)

Acquisition d'un flux de mesures par sentinelle, calculs cumulatifs et protection anti-crash.



Figure 2 : Chaîne d'acquisition de données par sentinelle et gestion du cas limite sans mesure.

Activité 3 — Saisie par sentinelle : Télémétrie optique 4 points

Mise en situation industrielle

Sur une chaîne automatisée de contrôle qualité, un **capteur de télémétrie laser optique** mesure en temps réel la distance séparant l'émetteur d'une série de pièces usinées défilant sur un convoyeur.

Chaque mesure valide correspond à une distance réelle positive $A \geq 0.0$ cm. La transmission s'arrête dès que le capteur émet la valeur sentinelle **-1.0** (fin de lot ou arrêt du défilement).

Patron d'acquisition par sentinelle

```
Lire(A)
nb_mesures ← 0
Tant que (A ≥ 0.0) Faire
  nb_mesures ← nb_mesures + 1
  Lire(A)
Fin Tant que
Écrire("Nombre de mesures : ", nb_mesures)
```

1. Pourquoi la variable **A** doit-elle être lue une première fois *avant* la boucle **Tant que**, puis à la *fin* du corps de boucle ?
2. Vérifier que ce patron d'écriture garantit que la sentinelle **-1.0** n'est **jamais comptabilisée** dans **nb_mesures**.

Activité 4 — Calcul cumulatif & Protection anti-division par zéro 4 points

Compléter le programme pour calculer la somme totale et la moyenne arithmétique des mesures valides :

1. Initialiser un accumulateur **somme** à 0.0 et l'actualiser à chaque tour de boucle avec la mesure valide courante.
2. **Sécurité algorithmique** : Que se passe-t-il si l'opérateur saisit immédiatement **-1.0** ? Pourquoi le calcul direct **moyenne = somme / nb_mesures** provoquerait-il un arrêt brutal (ZeroDivisionError) ?
3. Insérer la structure **if nb_mesures > 0: ... else: ...** afin d'afficher la moyenne ou le message **"Aucune mesure valide enregistrée"**.
4. **Jeux d'essais à valider** :
 - **Cas nominal** : 12.5 → 4.2 → 18.3 → -1.0
⇒ 3 mesures, Somme = 35.0, Moyenne = 11.67 cm.
 - **Cas limite (0 itération)** : -1.0
⇒ **"Aucune mesure valide enregistrée"** sans crash machine.

 **SYNTHÈSE DIDACTIQUE** : Le calcul de moyenne après sentinelle nécessite toujours une **protection conditionnelle préalable** (**nb_mesures > 0**).

✍ *Sur votre copie : Programmez les boucles, intégrez la protection et validez les jeux d'essais sur machine.*

4. Palier Dépassement & Excellence (4 points)

Modéliser la dynamique comparée de deux systèmes et réaliser la recette logicielle d'un capteur.

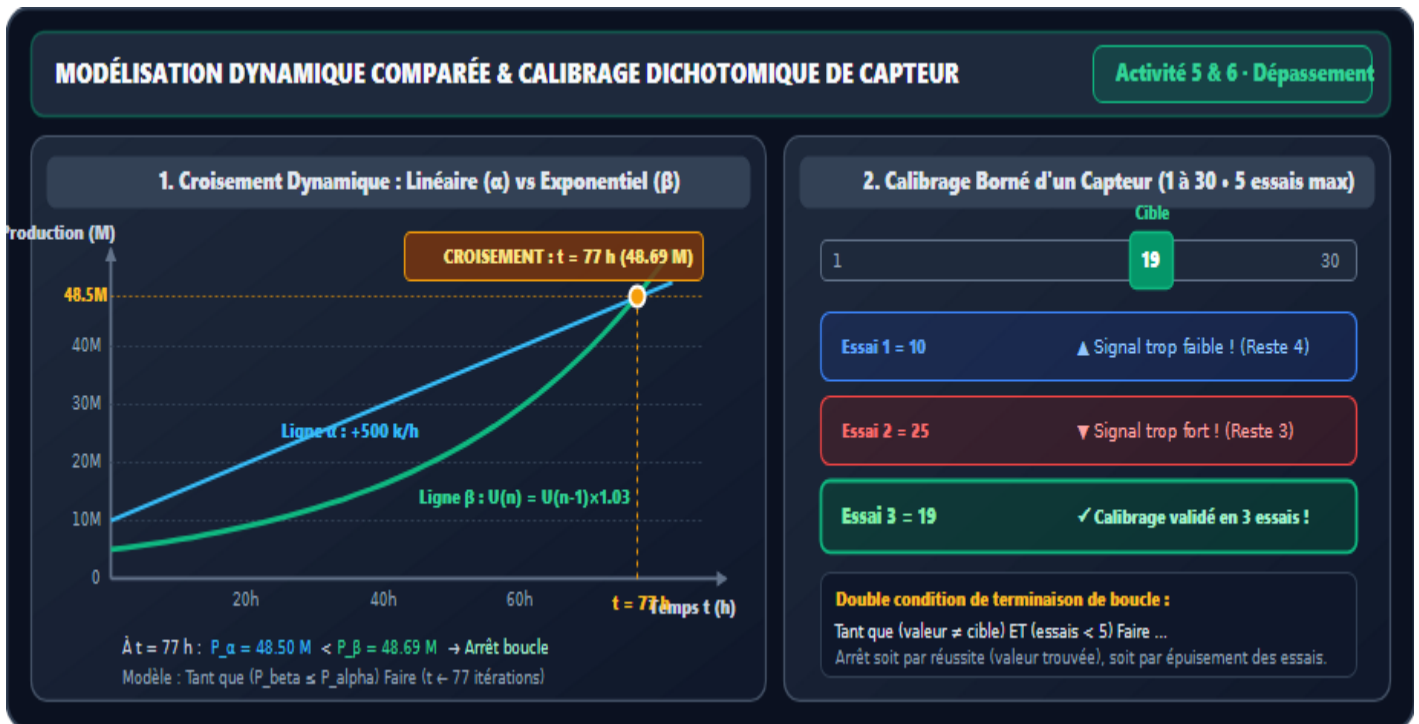


Figure 3 : Modélisation dynamique comparée linéaire vs exponentielle et procédure de calibration de capteur.

Activité 5 — Modélisation dynamique comparée : Lignes de fabrication Ex. 20 2 points

Mise en situation industrielle (Usine automatisée)

Deux lignes de fabrication robotisées produisent des composants électroniques :

- **Ligne α (Croissance linéaire) :** Production initiale $P_\alpha = 10\,000\,000$ pièces, cadence constante de $+500\,000$ pièces/h.
- **Ligne β (Croissance géométrique) :** Production initiale $P_\beta = 5\,000\,000$ pièces, gain continu de $+3\%$ par heure ($P_\beta \leftarrow P_\beta \times 1.03$).

Objectif : Déterminer au bout de combien d'heures de fonctionnement la production de la ligne β dépassera celle de la ligne α.

Jalons d'évolution horaire

Temps t	Ligne α (Linéaire)	Ligne β (Exponentielle)	$P_\beta > P_\alpha$?
$t = 0 \text{ h}$	10.0 M	5.00 M	Non
$t = 20 \text{ h}$	20.0 M	9.03 M	Non
$t = 40 \text{ h}$	30.0 M	16.31 M	Non
$t = 60 \text{ h}$	40.0 M	29.46 M	Non
$t = 77 \text{ h}$	48.50 M	48.69 M	OUI (Dépassement !)

1. Formuler la condition de maintien de la boucle **Tant Que** modélisant cette évolution : **Tant que** ($P_{\beta} \leq P_{\alpha}$) **Faire**.
2. Écrire l'algorithme complet **ProductionComparee** avec compteur d'heures **t**.
3. Implémenter et exécuter la simulation sous Python.
4. Vérifier qu'au bout de **77 heures**, la ligne β (48 689 611 pièces) dépasse la ligne α (48 500 000 pièces).

Sur votre copie : Rédigez l'algorithme sur votre feuille et enregistrez le script dans **TP1_Nom_Prenom.py**.

Activité 6 — Calibrage d'un capteur par encadrement & Recette Ex. 114 2 points

Procédure de réglage (Exercice 114)

Un capteur de position doit être calibré sur une valeur de consigne entière secrète comprise entre 1 et 30. Le technicien dispose de **5 tentatives maximum**. À chaque essai, le système indique "Signal trop fort !" ou "Signal trop faible !".

1. Identifier la double condition d'arrêt de la boucle : ($\text{valeur} = \text{cible}$) ou ($\text{essais} \geq 5$).
2. Écrire le script simulant le calibrage et afficher le message de succès ou d'échec au terme des 5 essais.

Grille de recette logicielle à faire valider par l'enseignant

N° Test	Activité & Données de test	Résultat attendu	Résultat machine	Visa Enseignant
Test 1	Activité 1 : $\text{code} = 5403$ (test préalable -12)	Rejet de -12 ; 5403 $\Rightarrow$ 4 chiffres (Technicien Régleur)		[] Validé
Test 2	Activité 4 : 12.5 $\rightarrow$ 4.2 $\rightarrow$ 18.3 $\rightarrow$ -1.0	Somme = 35.0 ; Moyenne = 11.67 cm		[] Validé
Test 3	Activité 4 : Saisie directe de -1.0	Alerte : Aucune mesure (anti-crash)		[] Validé
Test 4	Activité 5 : Modélisation α vs β	Croisement en 77 heures ($P_{\beta} = 48.69 \text{ M} > 48.50 \text{ M}$)		[] Validé
Test 5	Activité 6 : Calibrage capteur	Arrêt en ≤ 5 essais avec retours guidés		[] Validé

Sur votre copie : Faites valider vos 5 tests par l'enseignant et remettez votre copie ainsi que votre fichier **TP1_Nom_Prenom.py**.